A quick tour of Torch internals

Oct 18, 2015

Recently, I have been kind of confused. I couldn't find myself anything to work on and had no ideas for new projects (apparently, I just had to wait for the new academic year to start - I have plenty of ideas now, but no time for them).

Anyway, I often get the impression that many people are using Machine Learning libraries as a kind of black-boxes with only a high-level API. It's as if they weren't interested at all in how they work, but solely in the output (this is why I like Torch so much - it's hackable to the bone). I've been using Torch for a few months now and I've always been curious how it's built. This is why I decided to get down to it and browse the code of TH library, which is at the core of Torch.

It's really a great thing to do. I'll write more about it in the end of this post, but you should seriously consider doing it with your favourite library or framework too.

Torch's source is written in plain C, which was very pleasing to me. I don't really like many C++ features and although I find it very powerful and flexible, it often seems confusing. C's extremely minimal syntax allows you to read and quickly grasp what exactly happens at any moment. However, if C++ is the way to go for you, there is also a wrapper around TH called thpp.

Where can you get it?

You can find the `TH` library in two places:

- In its standalone repository (git subtree of torch7; outdated at the time of writing)
- In torch7 repository in `lib/TH` folder (always up to date)

The folder structure is very simple. There are some cmake tests and definitions in `cmake` directory while the code is located both in `generic` directory and at the repo root.

Interesting findings

Before going into the details and describing functionality implemented in individual files I'd like to point out some really cool techniques that I've found in the implementation.

Code generation

First thing that appeared really strange to me was that many files existed both in the root folder as well as in `generic`. If you opened them, you would quickly notice that copies in `generic` contain the actual code, while at the root they all look very similar. Here is `THStorage.c` for example:

```
#include "THAtomic.h"
#include "THStorage.h"

#include "generic/THStorage.c"
#include "THGenerateAllTypes.h"

#include "generic/THStorageCopy.c"
#include "THGenerateAllTypes.h"
```

Quite unusual for a `.c` file, right?

`THGenerateAllTypes` sounds interesting so i looked it up and this if what I've found:

```
#ifndef TH_GENERIC_FILE
#error "You must define TH_GENERIC_FILE before including THGenerateAllTypes.h"
#endif

#define real unsigned char
#define accreal long
#define Real Byte
#define TH_REAL_IS_BYTE
#line 1 TH_GENERIC_FILE
#include TH_GENERIC_FILE
#undef real
#undef accreal
#undef Real
#undef TH_REAL_IS_BYTE

#define real char
#define accreal long
#define Real Char
#define TH_REAL_IS_CHAR
#line 1 TH_GENERIC_FILE
#include TH_GENERIC_FILE
#undef real
#undef accreal
#undef Real
#undef TH_REAL_IS_CHAR
...
```

Which continued for a few more types. At first I was puzzled, but then I suddenly realized what it does and how brilliant this is! There are no templates in C, but objects like THStorage should be available for different types. It would be a terrible waste to repeat the same implementation with just a few words replaced and this is what this piece achieves! In generic files you can see variables of type `real` all over the place. At first it was obvious to me that it's probably a matter of some compile time optimizations whether it was chosen to be a float or a double, but apparently it's different - it allows code generation for many other types too!

Clever usage of macros also makes the generic files more readable. Take this example from `generic/THStorage.c`:

```
real* THStorage_(data)(const THStorage *self)
```

It looks nice, but what about name conflicts for different types? It can't be `THStorage` and `THStorage_data` all the time! Worry not, macros take care of that as well:

```
#define THStorage         TH_CONCAT_3(TH,Real,Storage)
#define THStorage_(NAME) TH_CONCAT_4(TH,Real,Storage_,NAME)
```

During preprocessing this function name will be expanded to something like `THByteStorage_data` and `THStorage` will be replaced with `THByteStorage`. Super cool x2!

It's also smart to use a `#line 1 TH_GENERIC_FILE` directive, because if there would be any errors they will appear in the compiler as if they were in the original generic file - not in the middle of the implementation pasted over and over.

I think that these are some awesome ways to make C code more type-agnostic.

OOP & Virtual tables

TH also implements a file API, where you can find good examples of how you could implement some basic OOP patterns in C. There are four files that I'll be talking about here:

- `THFilePrivate.h` - defines basic structs
- `THFile.c` - contains some generic implementation
- `THDiskFile.c` - code for handling disk files
- `THMemoryFile.c` - implementation of in-memory files

Let's start with the private header file.

```
struct THFile__
{
    struct THFileVTable *vtable;

    int isQuiet;
    ...
};

struct THFileVTable
{
    int (*isOpened)(THFile *self);

    long (*readByte)(THFile *self, unsigned char *data, long n);
    long (*readChar)(THFile *self, char *data, long n);
    long (*readShort)(THFile *self, short *data, long n);
```

```
    long (*readInt)(THFile *self, int *data, long n);
    ...
};
```

You can see that it defines a virtual method table with pointers to functions that `THFile` subclasses will have to implement (`THFile` is an abstract class - it has no constructors). Other structs are defined as such:

```
typedef struct THDiskFile__
{
    THFile file;

    FILE *handle;
    char *name;
    int isNativeEncoding;

} THDiskFile;
```

What makes this struct interesting is that because it's first member is of type THFile it's actually valid to cast `struct THDiskFile *` to `struct THFile *` and use it normally. What's more, because `THDiskFile` 's constructor fills in the function pointers in `file` field's virtual table, it will behave as `THDiskFile` object even when casted to `THFile` !

Shared memory

I had little knowledge about UNIX process management and threading until now, when I took up an operating systems course at my university, so it was really interesting to learn about `mmap` (maps a file to memory, so you can use it like an array) and to see how memory can be shared between processes with `shm_open` . I even wrote a piece of code to try it out. You can find it here.

SIMD

Another cool thing you can find in `TH` are vector instructions. There are some cmake tests that check if they are available on your CPU (`cmake/FindSSE.cmake`) and several files implementing convolution operations using them (`generic/simd/*`). I can't understand it yet - function that takes 10 lines of code is expanded to unrolled vectorized loop taking more than 120 lines and using APIs with unreadable function names for a SSE beginner. This code spans 134 lines after macro expansion:

```
void convolve_5x5_1_avx(float* output, float* image, float* weight, long count, l
  long i = 0;
  long alignedCount = count & 0xFFFFFFF8;
  DECLARE_OUTPUT_1()
  for (; i < alignedCount; i+=8) {
    CONVOLVE_8COLS_XROWS(1, i)
  }
}
```

Anyway, it's definitely a thing worth learning so I will probably write more about it soon!

Allocators

`TH` declares it's own function for memory allocation called `THAlloc`. It tries to allocate a properly aligned chunks if you allocate big blocks and handles out-of-memory errors. Before reading Torch's source I didn't know about the concept of allocators. They are just small virtual tables providing their own memory management API (alloc, realloc, free). It's cool that you can pass an Allocator to `THStorage` or `THTensor` and construct it not only in the regular heap region, but also allocate it in the shared memory.

Random module

It's natural to have a pseudorandom number generator in all programming languages, but I've never read an implementation of one (ok, except the linear congruential generator). In `THRandom.c` you can find a full implementation of Mersenne twister, which (according to Wikipedia) is a default implementation for R, Python, Ruby, PHP, CMU Common Lisp, GLib, MATLAB and some more. There are also several methods which convert returned uniform distribution into other shapes.

Quick library overview

In this section I will briefly describe most of the functionalities provided by `TH`.

- **THAllocator**
  - creates a default default allocator, which just calls `TH` memory management functions and, if possible, a `THMapAllocator` that can map files or shared memory objects into memory.
- **THAtomic**
  - multiplatform implementation of atomic operations
- **THTensor**
  - defines a general Tensor type
  - supports lots of indexing, linear algebra and math operations
  - available for all primitive datatypes (`TH<type>Tensor` , e.g. `THFloatTensor`)
- **THBlas**
  - wraps BLAS library for use in `THTensor`
  - provides a general implementation as a fallback
- **THLapack**
  - wraps LAPACK library for use in `THTensor`
  - **DOESN'T** provide fallbacks - throws errors if called
- **THFile**
  - abstract file class
  - only creates wrappers for calling methods contained in virtual table
- **THDiskFile**
  - concrete file class
  - wraps disk file APIs
- **THMemoryFile**
  - concrete file class

    - operates on an in-memory buffer and fakes file operations
- **THGeneral**
    - implements general utilities
    - contains memory management routines
    - can notify external GCs
- **THRandom**
    - implements a random number generator
    - can sample from many distributions
- **THStorage**
    - defines a general storage object
    - contains mainly bookkeeping code
    - available for all primitive datatypes (`TH<type>Storage` , e.g. `THFloatStorage`)

How to use it

If you want to install `TH` you can either perform a full Torch installation or you can follow these steps:

```
# clone Torch repository
git clone https://github.com/torch/torch7
mkdir th_build
cd th_build
# configure TH build
cmake ../torch7/lib/TH
# compile library
make
# install shared library and header files
make install
```

Then, you only have to `#include <TH/TH.h>` in your program and link the library during the compilation process (`-lTH`).

Example program

To wrap up I just wanted to show you an example program using `TH` . It will simply load 10 floats from two files into tensors, compute their dot product and add to it a sum of all values in one of them. This is the code:

```
#include "TH/TH.h"

int main()
{
    THFile *x_file = THDiskFile_new("x", "r", 0);
    THFile *y_file = THDiskFile_new("y", "r", 0);

    THFloatTensor *x = THFloatTensor_newWithSize1d(10);
    THFloatTensor *y = THFloatTensor_newWithSize1d(10);
```

```
    THFile_readFloat(x_file, x->storage);
    THFile_readFloat(y_file, y->storage);

    double result = THFloatTensor_dot(x, y) + THFloatTensor_sumall(x);

    printf("%f\n", result);

    THFloatTensor_free(x);
    THFloatTensor_free(y);
    THFile_free(x_file);
    THFile_free(y_file);
    return 0;
}
```

All input parsing and possible errors are handled by Torch. Convenient, isn't it?

Afterthoughts

I actually enjoy reading other's source code - especially if it's well written. If you have some spare time, then seriously, consider picking your favourite library or framework, and try to understand how it works - even the tiniest bits of it. I guarantee that you will find many fascinating things and learn many concepts and ways of structuring your code that you had no idea existed. I haven't learned that much in such short period of time for a while. I liked it so that I'm thinking about doing this on a more regular basis.

`TH` has no documentation at the moment. Since I've already studied most of it's code, I'll probably try to write at least a bit. I've used Torch for so long that it's time to make some contribution myself.

Thanks for reading! I hope that you liked it!

16 Comments Login

Join the discussion...

LOG IN WITH OR SIGN UP WITH DISQUS

Name

12 • Share Best Newest Oldest

Michael
9 years ago
Wow, this code is ugly. I find it absolutely insane that this isn't written in (modern) C++. Which would imply a lot less pain to write, a

lot less pain to look at the code, and probably fewer overall bugs, too.
Why on earth would you "emulate" templates or dynamic polymorphism in a very bad and hack-y way when you can simply have them with C++ at no extra cost? This is a serious question.

1 1 Reply



Adam Paszke Mod → Michael

9 years ago edited

I disagree. It's totally personal what's easier to read and write - I like C way more. TH could have been written in C++, but I can find many reasons that C is a better choice eg.:

1. Torch (and TH) is 13 years old and in the beginning it was written using C++, so it was probably a conscious decision to switch to C.
2. C is very good for open source projects because it's simple. You don't need people with several years of experience in C++ to maintain your project. Anyone can quickly grasp most of C features and contribute themselves.
3. It's easier to reason about performance when you're using C. This is a critical thing in scientific computing software and usage of the high-level features of C++ can make it harder to find and optimize bottlenecks. Is writing a BLAS library in assembly insane?
4. It's easier to connect a C interface to other programming languages.
5. I fell many C++ features are often overused and become totally unreadable for everyone except core contributors.
6. You get the benefits of small executable size and portability (not every platform has a modern C++ compiler), which allows You to easily embed it to various platforms (e.g. there is a wrapper for iOS).

Also, you can find this in ROADMAP.md: "...I could move to C++ if it was only template...

see more

2 0 Reply



Michael → Adam Paszke

9 years ago

1. It's likely that this decision was made by someone who does not know [modern] C++ very well. Or someone who first learned C, and did not unlearn the C-style ways when trying to switch to C++.
2. C is not actually simple; its implied simplicity is deceptive. It doesn't scale to large software 'cause you're stuck on a very low level of abstraction. It is arcane and allows you to blow off your foot. Of course you can do the same in C++ if you really want to (it being a pretty much a superset of C), but using the right style guidelines, you can write much easier and safer code in C++, at absolutely no expense of performance.
3. That's a myth. It is not harder to reason about performance when using C++ in the proper way.
4. True. That's why some people write hourglass interfaces (C++ API -> C wrapper -> C++ wrapper).
5. Not C++ specific. You can abuse any language and their features. The amounts of terrible C code abusing macros etc. out there are frightening. (And I count the above code among those.)
6. All reasonable platforms out there that I'm aware of, including mobile phone platforms, have a modern C++ compiler, or you can build one for them. Torch is

not targeting embedded DSPs, or is it?

Regarding the roadmap: So what? See 6), and C++11 has been standardized 5 years ago, and the support for it is pretty much ubiquitous. There is no reason to be afraid of advanced features.

1 1 Reply



chinso00 → Michael

9 years ago

> Why on earth would you "emulate" templates or dynamic polymorphism in a very bad and hack-y way when you can simply have them with C++ at no extra cost?

So that you can control linkage. Interfacing (especially via FFI) to other languages becomes infinitely more simple if you can control the linkage. C++ name mangling is a non-standardized mess.

> It's likely that this decision was made by someone who does not know [modern] C++ very well.
Just FYI, Torch3 was in C++, and we are not unaware of the great C++11, C++17 and C++9999 features.

> All reasonable platforms out there that I'm aware of, including mobile phone platforms, have a modern C++ compiler

(a) The CUDA compiler just got C++11 support in CUDA 7.5, less than 6 months ago.
(b) Well used linux distros such as CentOS 6.5 are still stuck on GCC

see more

3 0 Reply



Michael → chinso00

9 years ago

> So that you can control linkage. Interfacing (especially via FFI) to other languages becomes infinitely more simple if you can control the linkage. C++ name mangling is a non-standardized mess.

True, it's not standardized (yet), although there are efforts to make this happen. Controlling linkage, however, is not a problem through hourglass interfaces; see e.g. CppCon 2014: Stefanus DuToit "Hourglass Interfaces for C... – disq.us . Not a great solution, but solves the problem: One can write a beautiful C++ core and have an ugly C89 API on top.

> (a) The CUDA compiler just got C++11 support in CUDA 7.5, less than 6 months ago.

But that's only for CUDA code and not any other logic, right?

> (b) Well used linux distros such as CentOS 6.5 are still stuck on GCC 4.4x, yes.

It's quite trivial for those poor souls to build their own, shiny new GCC

see more

0 1 Reply



Adam Paszke Mod → Michael

9 years ago

C++ wasn't that modern 10 years ago. Somebody decided to stick with C back then and it affected the whole ecosystem. There are lots of packages that are dependent/extend Torch and they assume that it's written in C. The only C++ features that could replace things from this post are templates for two structures and one class… It's totally not worth rewriting the whole codebase just for them.

Actually C is very simple (especially compared to C++). You can learn the whole language in a moment. Only macros are sometimes a bit more advanced, but these used in here are pretty straightforward. There are many more people who can understand C than C++ and it's easy to reason about C++ performance only if you know it very well. Please remember that not everyone is a C++ ninja.

I understand that you cannot accept the usage of techniques presented in this post, but this doesn't make them insane. Actually, as far as I see, there are many people who like them. I just wanted to highlight cool things that can be done with C, I think that you might have missed the point.

I'm sorry but I can't see what benefits would switching to C++ bring for Torch.

2 0 Reply



SoulFireMage → Adam Paszke

2 years ago

You've almost persuaded this non c and non c++ user to consider learning c. I'm a student of Tensor and current dot net coder (hold the laughs at the back there). I'm going to learn pytorch this year, but contemplating now whether to learn C sufficient to follow this the way you have - especially as its more for interest than commercial reasons.

Thank you - 8 years later 😊

0 0 Reply



Cédric Deltheil → Michael

9 years ago edited

The vtable part is kind of classic - e.g. this is a pattern used in the Linux Kernel (see Object-oriented design patterns in the kernel, part 1, sqlite and many more extremely well designed C softwares). For the template part you can see this is currently being discussed in the Torch roadmap (there are definitely pros/cons and to some extent it is a matter of style/taste/readability).

1 0 Reply

Michael → Cédric Deltheil

9 years ago

I wouldn't call this "extremely well designed" -- it's a necessary crutch, and it's as good as you get in C, admittedly. Using C++, you get built in language features that do the same thing in better ways, and thus allow you to focus on expressing your code on a higher level abstraction than trying to emulate a feature that C is sorely missing.

0 1 Reply



Bombs Kim → Michael

7 years ago

You are a bigot

0 1 Reply



250mya

8 years ago edited

I also want to understand the internals of torch and pytorch. For me, it is difficult to understand how float and double can be derived from real. Anyway, thanks for this informative article!

0 0 Reply



Adam Paszke Mod → 250mya

8 years ago

When you write `real` and have a macro defined with that name, the compiler (in fact the preprocessor) will replace it (just as if you did find/replace) with the value of that macro. In this case, the value will be sometimes set to double and sometimes to float

0 0 Reply



250mya → Adam Paszke

8 years ago edited

What is confusing me is that in header file, real is defined as unsigned char and undefined, using #undef. Later, real is defined as char and undefined and so on. I am not accustomed to this grammar. How does it work?

In C++, it is difficult to use template when we have the header file and the other implementation source file. So I want to learn this kind of trick.

0 0 Reply



Ashis Samal

9 years ago

Thanks for sharing it

0 0 Reply



Cédric Deltheil

9 years ago

Nice post :) Some other areas that might be interesting to cover in future posts (or simply to dig into!) are luaT (a.k.a. the Lua Torch C API), the auto-generation of Lua interfaces by TensorMath.lua thanks to cwrap (even though this will probably evolve: see the roadmap) or everything related to the

serialization format.

0 0 Reply

A **Adam Paszke** Mod → Cédric Deltheil

9 years ago

Thanks! I'll check some of them for sure! :)
I wanted to read more about the serialization format, but I couldn't find any docs for it, and I wanted to see how TH works first. I'll probably write something about vector intrinsics in the next post, but if I find more time, then these topics may appear my blog as well!
Cutorch looks interesting too!

0 0 Reply

Subscribe Privacy Do Not Sell My Data DISQUS

apaszke
apaszke